
On the Limits of Innate Planning in Large Language Models

Charles Schepanowski
Western University
cschepan@uwo.ca

Charles Ling
Western University
charles.ling@uwo.ca

Abstract

Large language models (LLMs) achieve impressive results on many benchmarks, yet their capacity for planning and stateful reasoning remains unclear. We study these abilities directly, without code execution or other tools, using the 8-puzzle: a classic task that requires state tracking and goal-directed planning while allowing precise, step-by-step evaluation. Four models are tested under common prompting conditions (Zero-Shot, Chain-of-Thought, Algorithm-of-Thought) and with tiered corrective feedback. Feedback improves success rates for some model-prompt combinations, but many successful runs are long, computationally expensive, and indirect. We then examine the models with an external move validator that provides only valid moves. Despite this level of assistance, none of the models solve any puzzles in this setting. Qualitative analysis reveals two dominant deficits across all models: (1) brittle internal state representations, leading to frequent invalid moves, and (2) weak heuristic planning, with models entering loops or selecting actions that do not reduce the distance to the goal state. These findings indicate that, in the absence of external tools such as code interpreters, current LLMs have substantial limitations in planning and that further progress may require mechanisms for maintaining explicit state and performing structured search.

1 Introduction

In recent years, Large Language Models (LLMs) have demonstrated state-of-the-art performance on an expanding range of tasks. Successive models exhibit increasingly sophisticated capabilities [1, 2], yet their evaluation still centers largely on mathematics and code generation benchmarks [3–5]. This narrow focus leaves the dynamic, multi-step processes of reasoning and planning under-examined. When these abilities are studied directly, progress is often far less convincing than in other domains [6–8]. These limitations hinder the deployment of LLMs in complex, real-world applications that demand robust planning and state tracking, such as autonomous agents [9].

Although LLMs can often solve such problems by writing or calling basic search code, our goal here is to isolate their intrinsic planning and state-tracking abilities without such tools. We therefore ask a concrete question: how well can LLMs plan and reason over an evolving state when they cannot rely on code execution or other external tools? To answer this, we use the standard 8-puzzle and ask models to solve random configurations of the game. The 8-puzzle requires models to maintain an accurate representation of the board, obey strict move validity constraints, and choose moves that eventually reach a goal state. We measure not only whether a puzzle is solved but also how and why runs terminate when they fail, to understand the limits of model performance on this type of task.

We investigate the performance of four models—GPT-5-Thinking, Gemini-2.5-Pro, GPT-5-mini, and Llama 3.1 8B-Instruct—on 8-puzzle instances with varying optimal solution lengths, using three common prompting strategies: Zero-Shot, Chain-of-Thought [10, 11], and Algorithm-of-Thought [12]. Under each prompting strategy, each model receives a single attempt per puzzle. We then further assist the models on puzzles they initially fail to solve by allowing three additional attempts per level

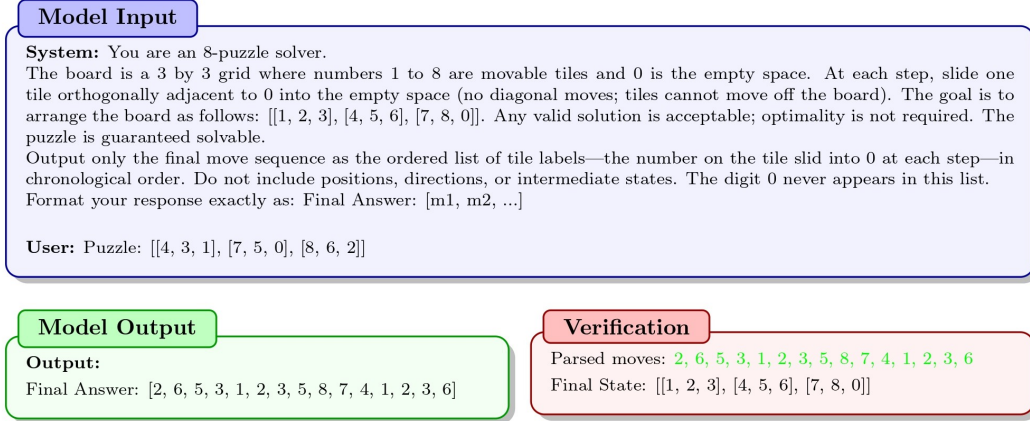


Figure 1: Example of our 8-puzzle evaluation pipeline. The model receives the system prompt, in this case, our Zero-Shot prompt, as well as the puzzle. The model then returns a sequence of moves. An external function applies each move to determine whether the model solved the puzzle. We highlight the legal moves in green. In this example, the model solves the puzzle on its first attempt.

of feedback (repeat, specific, and suggestive). Within each feedback level, we save progress between calls. With these additional attempts, we observe more success under some conditions for particular models (GPT-5-Thinking with AoT and suggestive feedback). However, no single approach performs best across models, and the improvements are slow and costly. The final intervention we investigate is an external move validator that offloads the burden of determining move validity. The models no longer have to decide which moves are valid and which are not; we provide them with a list of all valid moves and ask them to return the best one. We then apply the move, update the board state and list, and reprompt the model. Despite this assistance, no model solves any puzzles.

Across models and conditions, two dominant deficits emerge: fallible representations of the board state, leading to invalid moves, and weak heuristic planning, leading to loops or moves that do not advance the puzzle toward the goal state. This work moves beyond aggregate performance metrics to offer a granular, qualitative analysis of why LLMs struggle with such tasks and how different interventions modulate these failures. Our contributions can be summarized as:

- A tool-free 8-puzzle evaluation across four LLMs under Zero-Shot, CoT and AoT prompting and tiered feedback, with a common protocol that isolates the effects of prompting versus feedback.
- A fine-grained failure-mode analysis with per-condition breakdowns, revealing how and why models terminate instead of only reporting success rates.
- An external move validator condition that eliminates the need to determine which moves are valid, isolating planning capabilities and quantifying each model’s ability to work toward a goal when state tracking is no longer the bottleneck.

2 Related Work

2.1 Improving LLM Reasoning and Planning

Many studies have investigated how to improve LLM reasoning without fine-tuning models [10, 13, 14, 12]. A common approach that consistently shows improvement is prompt engineering, where one uses the system prompt to specify how the model should approach a problem. Chain-of-Thought [10, 15] is a familiar example, and it often asks the model to work step-by-step through a task. Algorithm-of-Thought [12] is another approach that instructs the model to think like a search algorithm. Other works seek to decompose the problem [16, 14], allowing the model to explore several reasoning branches and then proceed with the most promising one based on a heuristic such

as self-critique. Notably, even with guidance on how models should approach the problem, these methods still rely on the models’ own capabilities to complete the task.

However, other work has offloaded some of the burden from the models when solving problems. For example, some works improve LLM performance by using tools such as code interpreters, which offload part of the computation to a separate program [17, 18]. Additional work improves model performance by enabling models to access the internet and consult resources such as Wikipedia [19, 20]. With these extra tools, the models often outperform prompt-only methods. However, their performance can no longer be attributed solely to the models themselves, as it now depends on external information and computation.

2.2 Evaluation Methods

LLM progress is often measured based on benchmark performance, such as AIME [21, 4], SWE-bench [5], or others [22–24], highlighting their proficiency in mathematics, writing code, or other complex tasks. These benchmarks often rely on arithmetic or knowledge-based questions, where success is determined by recalling facts or using similar examples from the training data to solve the problems presented. However, many studies have examined the possibility of data contamination [4, 25, 26], raising concerns about the validity of the improvements we observe across models. Furthermore, these tasks offer less insight into the models’ planning abilities because they rarely require a long-term strategy.

Work that focuses on LLM planning often reports low success rates and provides high-level analyses of why models fail—for example, prompt-sensitive pattern-matching and the need for planner-based repair [27]. Other recent evaluations include cost/efficiency comparisons with some failure-mode breakdowns [6, 28], but they do not typically emphasize fine-grained analyses of how failures unfold. Furthermore, few studies examine variable levels of feedback, and we are aware of none that systematically combine modern prompting techniques with feedback on a planning task like the 8-puzzle to analyze how these factors modulate performance and failure types, leaving the effects of prompting and feedback on such tasks under-examined.

3 Methods

3.1 8-puzzle

The 8-puzzle is a simple and widely used search problem in AI [29, 30]. The puzzle consists of a 3-by-3 grid with eight numbered, sliding tiles (1–8) and a single blank space (represented as 0). Tiles directly adjacent to the space can slide into it to change the board configuration. From a random starting configuration, the goal is to slide tiles until the board reads 1–8 from left to right, top to bottom, with the blank in the bottom-right corner. The 8-puzzle is particularly well-suited to our investigation because it is a well-studied problem in AI [29], it provides a controllable and predictable environment in which the model must track the board state and obey strict rules, and it demands non-trivial long-term planning because solutions are unlikely to be found through local or random actions alone. Thus, it combines state tracking and planning in a single, compact domain, with easily verifiable solutions and unambiguous instructions.

The problem space consists of 181,440 unique solvable configurations, with the shortest optimal solution requiring zero moves and the longest optimal solution requiring 31 moves [30]. To ensure fair representation of puzzles with varying difficulty, we divided them into difficulty bins based on their optimal solution lengths. Each bin represents roughly 20% of the solvable puzzles and contains 10 sampled puzzles (see Fig. 3 for the bin definition). We generated the puzzles for each bin by creating a flattened list of integers from 0 to 8, shuffling them with Python’s `random.shuffle`, and reshaping them into 3-by-3 arrays. For each bin, we repeatedly sampled random boards and used only those that were solvable according to the standard parity test. Given the large number of possible puzzles and their random generation, we minimize the likelihood of data contamination.

3.2 Models

We tested four models in our experiments: GPT-5-Thinking, Gemini-2.5-Pro, GPT-5-mini and Llama 3.1 8B-Instruct. To ensure uniformity and fairness between models, we used the same parameters

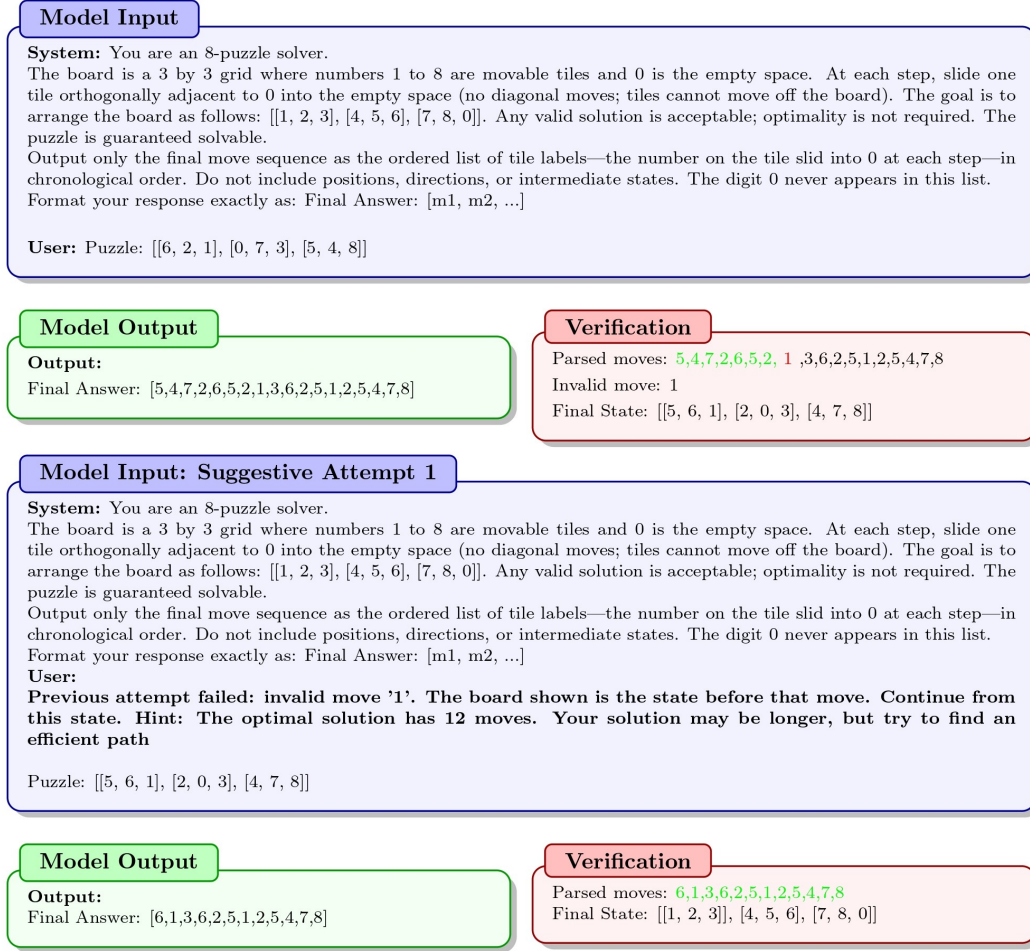


Figure 2: Example of our feedback pipeline. As in Fig. 1, the model first receives the Zero-Shot system prompt and the puzzle, but its initial attempt fails due to an invalid move (highlighted in red). We go back to the preceding valid state and re-prompt the model with suggestive feedback (shown in bold), after which the model successfully solves the puzzle.

and prompts for all four models: a token limit of 64,000, a time limit of 1,800 seconds per response, a temperature of 1.0, and identical system prompts. We set the temperature to 1.0 because the GPT-5 models do not allow variation from this value. Furthermore, a temperature of 1.0 is consistent with previous work [28], and even when variation is allowed, temperatures of 0.0 are not always deterministic with OpenAI models [6]. To control API costs, especially for the larger models, we evaluated all models on the same set of 50 puzzles, consistent with prior work [28].

3.3 Prompting Strategies

We study three prompting regimes under a common setup: **Zero-Shot** (no examples or reasoning cues), **CoT** (three worked examples encouraging stepwise reasoning), and **AoT** (three examples implementing a simple Manhattan-style search). The CoT and AoT prompts use the same three example boards, differing only in the reasoning styles they demonstrate. These three approaches serve as our *base conditions*, on which we build all subsequent attempts with and without feedback.

Zero-Shot: Our Zero-Shot prompt provides only the 8-puzzle rules, goal state, and output format, without examples or reasoning cues like "think step-by-step" [31]. This setup isolates the model's

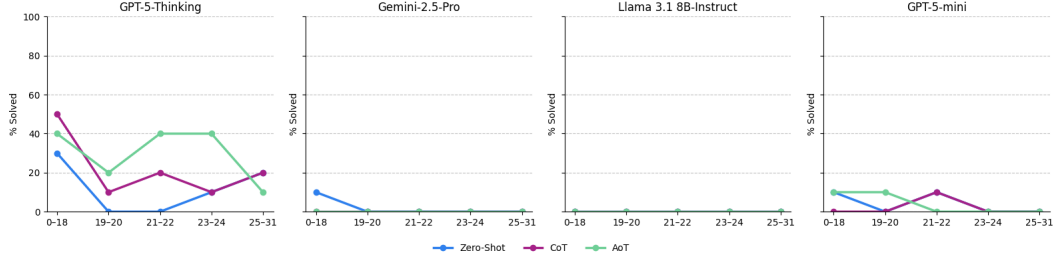


Figure 3: Success rates for each model under different prompting strategies across difficulty bins defined by optimal A* solution length. Each bin contains ten puzzles, and the percentage solved is computed within each bin.

baseline reasoning and planning abilities and serves as a control to benchmark unguided performance against CoT and AoT interventions. We illustrate this setup in Fig. 1.

Chain-of-Thought (CoT): CoT is a prompting approach that often enhances LLM problem-solving by instructing the model to generate intermediate steps [10, 11, 15]. We provided additional assistance using the few-shot paradigm, designing a 3-shot CoT prompt that gives the model a diverse set of worked examples. We chose three examples to limit prompt length and because prior work suggests that performance gains tend to diminish beyond a few examples [10, 31]. Our selected examples feature diverse board configurations and, critically, showcase complete solutions of varying lengths—3, 5, and 6 moves—and slightly different strategies to teach the model flexible and adaptive reasoning. The full prompt is available in Appendix A.2.

Algorithm-of-Thought (AoT): Algorithm-of-Thought works by guiding a language model to follow explicit algorithmic reasoning steps—often structured like pseudocode or formal rules—so that its outputs mimic systematic, step-by-step problem solving rather than free-form reasoning [12]. The original work reports improvements across many tasks and shows that AoT can outperform other prompting techniques, such as Chain-of-Thought (CoT) [10] and Tree-of-Thought (ToT) [14, 12]. For our work, we specify a simple Manhattan-style search—similar to A*—and again provide the same three example boards to stay consistent with the CoT prompt and ensure a fair comparison between approaches. For further details on the AoT prompt, see Appendix A.3.

3.4 Feedback

For each puzzle that a model failed to solve on its first attempt under any condition (Zero-Shot, CoT, or AoT), we ran three independent feedback trials: a repeat control, a trial with specific feedback, and one with suggestive feedback, as shown in Fig. 2. All three feedback trials start from the same failure state and have up to three additional attempts to solve the puzzle. That is, within a given trial, the attempt $k+1$ resumes from the **furthest valid state** reached in the attempt k . Concretely, if model M failed puzzle i under AoT (even if it passed under Zero-Shot or CoT), we re-prompted M with the AoT system prompt for up to three attempts under each feedback condition: repeat (no feedback), specific feedback, and suggestive feedback.

By *furthest valid state* we mean the last board configuration before failure: for `invalid_move`, where the model moves a tile off the board or one that is not adjacent to the space, or `loop_detected`, where the model repeats a board configuration, we resume from the state immediately before the offending move. If the model fails due to an `early_stop`, where it makes only valid moves but stops before reaching the goal state, we resume from the state after the last move. We reset the visited-state set at the start of each attempt. Attempts stop early upon success; otherwise, they terminate due to a timeout, token limit, early stop, loop, invalid move, or graceful failure, in which the model either refuses to solve the problem or asks for clarification on how to do so. We cap retries at three per feedback condition (repeat, specific, and suggestive) to align with prior work on iterative LLM correction [32] and to limit API costs. Across feedback conditions, we keep the system prompt and hyperparameters fixed.

Repeat: As a control condition, we re-prompt the model from the furthest valid state of its last failed attempt without providing any feedback. The prompt presents this state as a new, standalone puzzle. This trial serves as a baseline to measure performance gains arising purely from model stochasticity and from an additional attempt, thereby isolating the effects of the feedback provided under other conditions.

Specific: Specific feedback also resumes from the furthest valid state, but provides precise details on the previous failed attempt. If the model makes an invalid move, we inform it of the error and identify the specific tile involved. If it exceeds the time limit, we specify the exact duration allowed. We designed this level to help the model avoid immediately repeating the same mistake and to highlight where it went wrong so it can correct its behaviour. For example, highlighting that the model made an invalid move should encourage it to focus more on tracking the state space.

Suggestive: Suggestive feedback includes all elements of Specific feedback, plus a scalar hint: the *optimal solution length* for the current board. We computed this value exactly using Iterative Deepening A* (IDA*) with the admissible Manhattan-distance heuristic. Because the Manhattan heuristic is admissible (and consistent) for the 8-puzzle, the returned length equals the true optimum. We reveal only the length x —never the action sequence—and append: “Hint: The optimal solution has x moves. Your solution may be longer, but try to find an efficient path.” The hint provides a progress signal while still leaving planning to the model, aiming to prevent it from undertaking an unnecessarily long search. We illustrate this level of feedback in Fig. 2.

3.5 External Move Validator

In our final intervention, we offload part of the state-tracking burden from the model. The model receives the current puzzle state along with two lists: one containing all valid moves from the current position and another containing the previous move to help it avoid loops (e.g., immediately reversing the last action). This setup removes the need for the model to determine which moves are valid; it only has to examine the puzzle and select the best next move from the list. We instruct the model to return a single move, which we then apply to the puzzle. After applying the move, we update the puzzle state and move lists and re-query the model. Since the longest optimal solution for any 8-puzzle instance is 31 moves and roughly 94% of puzzles require 26 or fewer moves, we allow the models to take up to 50 moves per puzzle, providing ample opportunity to solve it while avoiding excessively long, run-on trajectories.

4 Results

4.1 LLMs Continue to Struggle

Initial experiments established a performance baseline across all four models using Zero-Shot, CoT, and AoT prompting. The results, summarized in Fig. 4, reveal that successful puzzle completion was rare for most models. GPT-5-Thinking was the only model to achieve a modest success rate, with 30% of puzzles solved under the AoT condition. This result represents the highest performance recorded in this phase, outperforming the next-best attempt (its own CoT performance) by eight percentage points.

In contrast to GPT-5-Thinking, the other models struggled significantly. Gemini-2.5-Pro and Llama 3.1 8B-Instruct were almost entirely unsuccessful, with Gemini solving only a single puzzle under the Zero-Shot condition and Llama solving none. The primary failure mode of these models is invalid moves across all prompting strategies, except Llama with CoT, which had a spike in parse errors because there was no list to extract from its answers. Interestingly, GPT-5-mini outperformed Gemini-2.5-Pro by solving more puzzles overall and at least one puzzle under each condition. GPT-5-mini also demonstrated a notable divergence from GPT-5-Thinking; its Zero-Shot performance was comparable to GPT-5-Thinking’s at an 8% success rate. However, unlike its more capable counterpart, introducing more complex CoT and AoT prompts did not improve performance, as its success rate dropped under both approaches. With GPT-5-mini, we also see, in both CoT and AoT, a sharp increase in graceful failure, where the model refuses to continue or requests clarification.

Reviewing success rates by difficulty, as shown in Fig. 3, we observe that success does not decrease monotonically with optimal solution length. Although all models struggle with the longest puzzles,

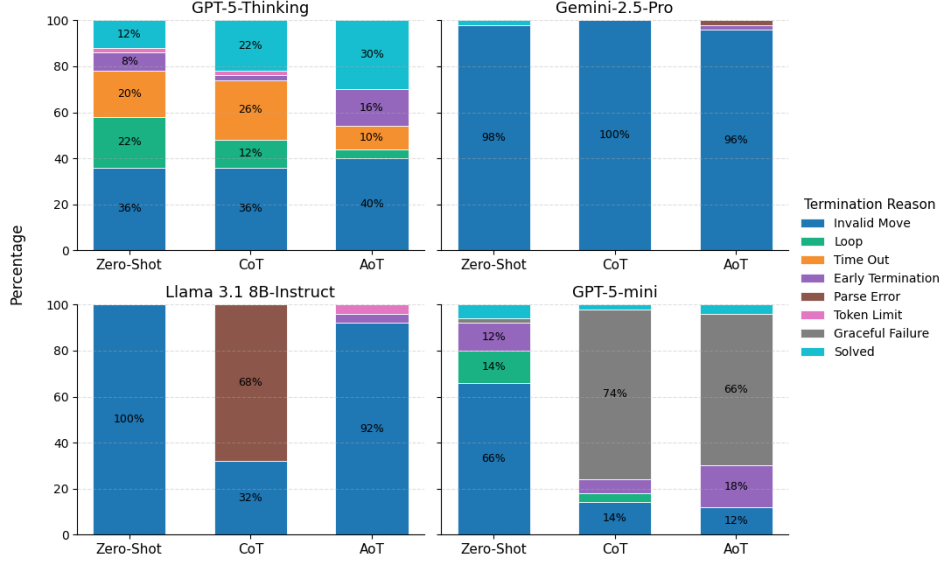


Figure 4: Termination breakdown of the four LLMs on the 8-puzzle across prompting strategies. Each stacked bar represents 100% of trials for a given model and prompting condition (Zero-Shot, CoT, or AoT). The colored segments show the percentage of trials that ended in success (Solved) or in each failure mode.

some achieve or match their best performance in the moderate-length bins. For example, GPT-5-Thinking with AoT matches its lower bin performance in two of the moderate bins. Similarly, GPT-5-mini, with its CoT prompt, solves no puzzles in the two shortest bins, then jumps up to 10% in the middle one. This pattern points to planning and state-tracking issues, rather than puzzle difficulty alone, as the main determinants of success rates.

4.2 Feedback Helps

As shown in Fig. 5, providing feedback, additional attempts, and saving progress yielded performance gains across all models and prompting strategies, except for Llama 3.1 8B-Instruct. GPT-5-Thinking showed the most significant improvement, with AoT prompting and suggestive feedback yielding a 68% success rate, the best performance for any model. As the level of feedback increased, GPT-5-Thinking’s performance improved with AoT and Zero-Shot, but the opposite occurred with CoT: its performance steadily decreased. We observe the same trend for GPT-5-mini under CoT. These results suggest that more informative feedback may not always be desirable for these models.

The other models, excluding Llama 3.1 8B-Instruct, also showed steady gains. Both Gemini-2.5-Pro and GPT-5-mini achieved a peak success rate of 18%. Notably, both models achieved this peak under the "Repeat" condition (Gemini-2.5-Pro with Zero-Shot and GPT-5-mini with CoT), suggesting that the primary benefit came from saving progress and re-prompting rather than from the feedback content itself. Gemini-2.5-Pro, however, matched this 18% performance when using the combination of AoT prompting and suggestive feedback. GPT-5-mini with CoT, similar to GPT-5-Thinking with CoT, shows a steady decline in performance as the level of feedback increases, again highlighting that more informative feedback is not always beneficial. Finally, at the lowest end of the performance spectrum, Llama 3.1 8B-Instruct failed to solve any puzzles even with feedback, underscoring a significant capability gap on this task.

4.3 External Move Validator Highlights Planning Deficits

In the final experiment, we aimed to isolate planning abilities by offloading part of the state tracking responsibilities. The results, shown in Fig. 6, reveal a critical failure in heuristic planning: none of the models solved any puzzles. The dominant failure mode for the models shifts to looping, in which they repeat a move and return to an already visited board configuration. GPT-5-Thinking looped in

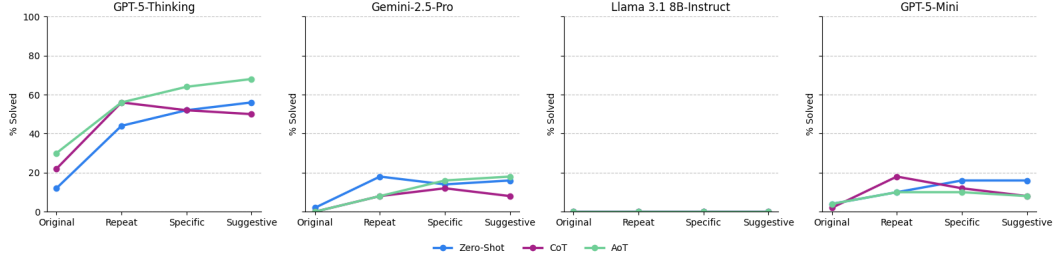


Figure 5: Success rates of each model under different feedback conditions. The “Original” point represents the initial success rate for each prompting strategy (Zero-Shot, CoT, and AoT). The “Repeat,” “Specific,” and “Suggestive” points show the final success rates after models were given up to three additional attempts on previously failed puzzles with the corresponding type of feedback. Each line tracks the performance of a prompting strategy across feedback conditions.

100% of trials, with Gemini-2.5-Pro and Llama 3.1 8B-Instruct looping 92% and 86% of the time, respectively. These results suggest that even when presented only with valid moves and the previous move to help track progress, these models struggle to formulate a goal-oriented path.

GPT-5-mini showed a different kind of limitation: it did not solve any puzzles, but its primary outcome was early termination (68%), meaning it consistently made valid moves until it reached the 50-move limit. Therefore, GPT-5-mini exhibits a different kind of planning deficit: it can make valid moves and track the state space—contrasting with its earlier single-prompt results in Fig. 4—but lacks a coherent long-term strategy, effectively wandering the state space without direction.

5 Discussion

5.1 What did we learn?

Our experiments reveal that a diverse range of LLMs, differing in producer and size, continue to struggle with simple planning and search-based tasks such as the 8-puzzle, even SOTA models like GPT-5-Thinking, which has been described as possessing PhD-level intelligence [33]. Models consistently fail due to two primary deficits: brittle internal state representations that lead to invalid moves and weak heuristic planning that results in loops. These deficits indicate that models often fail to maintain accurate board state representations or to devise effective long-term strategies. This finding underscores a key limitation: while LLMs increasingly excel on academic benchmarks, they remain challenged by sequential tasks that demand sustained planning and state awareness.

The prevalence of loop failures is especially revealing because the system prompt explicitly instructs models to avoid repeating moves. Despite this explicit constraint, all models frequently violated the instruction, suggesting a fundamental difficulty in adhering to constraints. This deficit is most apparent in the external move validator experiment: even when given a list of valid moves, an explicit instruction not to repeat moves, and their previous move, the models continued to loop. One might argue that the model’s internal heuristic changes between calls, but even then, it could simply select a different valid move when its preferred choice violates the constraints. Instead, it repeats moves, indicating that the planning process relies not only on weak heuristics but also fails to integrate the prompt’s explicit rules.

5.2 Effects of Prompting

Our experiments suggest that the utility of advanced prompting strategies is model-dependent, as illustrated in Fig. 4. While Algorithm-of-Thought (AoT) and Chain-of-Thought (CoT) improved performance for GPT-5-Thinking, they degraded performance for GPT-5-mini and Gemini-2.5-Pro. These findings demonstrate that, for this task, the efficacy of a prompting technique is contingent on the model, with no single approach proving universally superior.

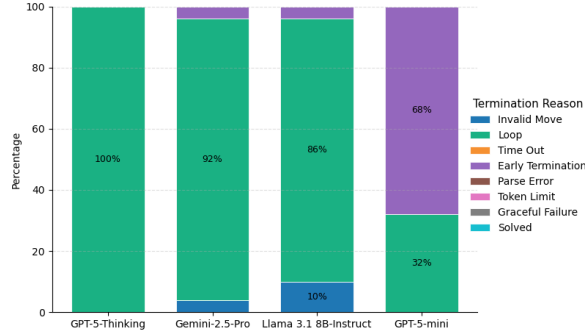


Figure 6: Termination breakdown of each LLM when using the external move validator. Each stacked bar represents 100% of trials for a given model. The colored segments illustrate the percentage of trials that terminated due to each failure type. The color scheme is the same as the one used in Fig. 4

The effects of prompting extended beyond success rates, causing significant shifts in termination modes (Fig. 4). For instance, Llama 3.1 8B-Instruct under the CoT condition exhibited an increase in parse errors, indicating a failure to adhere to the specified output format. Concurrently, GPT-5-mini showed a dramatic increase in graceful failure with both CoT and AoT, in which the model refuses to provide a solution or requests clarification. These outcomes suggest that, for some models, introducing complex prompts creates confusion, hindering their ability to follow instructions rather than aiding their reasoning.

5.3 The Cost of Feedback

GPT-5-Thinking saw the most significant performance increase with the addition of feedback and saved progress between calls. Its success rate increased from 30% to 68%, but this comes at a substantial cost. As Table 1 shows, the model needs, on average, two attempts, 24 minutes, over twice as many moves and 75,000 tokens to solve a puzzle. These averages are all higher when we include the puzzles that the models failed to solve. Furthermore, we obtain these results under carefully engineered prompts designed to be as helpful as possible for this task. Thus, while performance may increase, the compute and human resources required to provide these gains are substantial and problem-dependent. Gemini-2.5-Pro and GPT-5-mini both have similar issues with resource usage. While their average time, tokens and moves needed are lower overall, they require more attempts and have a substantially lower success rate of 18%.

Model	Approach	Feedback	Solved N	Time (min)	Tokens	Attempts	Moves	Optimal
GPT-5-Thinking	AoT	Suggestive	34	23.92	75284	1.97	48.91	21.03
Gemini-2.5-Pro	Base	Repeat	9	6.74	58651	2.67	30.56	19.67
Llama 3.1 8B-Instruct	Base	Repeat	0	0	0	0	0	0
GPT-5-mini	CoT	Repeat	9	9.91	44272	2.67	28.44	19.11

Table 1: Solved count and average time (minutes), tokens used, attempts, moves, and optimal moves for each model under its best-performing condition. We broke ties for GPT-5-mini and Gemini-2.5-Pro by choosing the approach that required the fewest tokens.

5.4 Planning with an External Move Validator

In our final experiment with the external move validator, no model solved a single puzzle. As shown in Fig. 6, this intervention hurt performance, lowering the success rates of all models and shifting the dominant failure mode to looping in most models and early termination in GPT-5-mini. Fig. 7 further illustrates the planning deficit, showing that the models made minimal progress toward the goal state despite making numerous valid moves. For instance, GPT-5-mini made, on average, nearly forty-seven valid moves to achieve only about six moves of progress based on a Manhattan-distance heuristic. GPT-5-Thinking and Gemini were more efficient in that they made better progress in fewer moves, but this matters little, as they often began to repeat moves, completely stagnating progress.

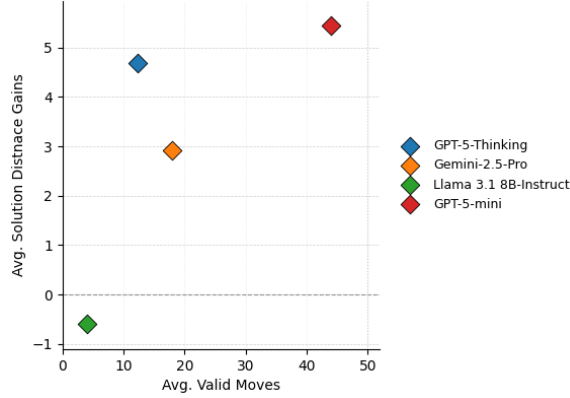


Figure 7: The average number of valid moves a model made with each puzzle vs the average distance gain from the solution based on the initial and final state using Manhattan distance.

This finding highlights that, even when given only valid moves, models either rely on heuristics so weak that solving a typical puzzle could require well over 100 moves, or they begin repeating moves and cycling through already visited states.

5.5 Implications

Our findings, though derived from a single domain, carry significant implications for LLM deployment. We observe that all models struggle with this simple task, and while engineered prompts with feedback and saved-state progression show improvement, they are costly. These interventions also fail to make any model reliable, as none achieves a success rate over 70%. The nature of these failures is also concerning, with the primary deficits not just weak heuristic planning that leads to loops, but also a persistent difficulty in maintaining an accurate representation of the board state, leading to "hallucinated" or invalid moves. These errors lead to a critical issue: models frequently output solutions as if they have correctly solved the puzzle, without detecting the mistakes made earlier in the sequence. This kind of false confidence undermines trust, because a model’s final output may rest on undetected errors earlier in the sequence.

These findings challenge the vision of deploying LLMs as autonomous agents in complex, real-world applications. If an LLM cannot reliably track its own state in a simple, deterministic puzzle, it may have similar issues in high-stakes, dynamic environments. In domains such as autonomous robotics, financial trading, or logistics management, an agent that proceeds with high confidence after an internal error can be unreliable and potentially dangerous. It risks costly or harmful outcomes by acting on a flawed premise.

5.6 Absence of Coding Tools

One might ask why we do not simply allow the models to use code interpreters for this task—after all, using tools is itself a form of reasoning. However, allowing the model to write and execute code changes the nature of the problem: the evaluation then focuses on whether the model can produce valid code, a capability already well studied by existing benchmarks. Moreover, code-based search largely masks planning and state tracking, since an external algorithm can solve the puzzle without the model needing to devise or maintain a long-term strategy. Therefore, in our experiments, we exclude code interpreters because they would mask the abilities we seek to examine.

6 Conclusion

In this work, we examined a range of LLMs using the 8-puzzle to probe their planning and stateful reasoning capabilities. Our findings show that both SOTA and small models still struggle significantly with this seemingly simple task, with failures indicating critical deficits in state tracking and heuristic planning. While feedback and prompt engineering offer some help, they are costly and do not provide

the gains needed to make these models reliable. Furthermore, even when offloading some of the burden by providing the model with valid moves and its last move, no model solves any puzzles. The tendency for models to be unaware of their mistakes and to proceed with high confidence after undetected errors poses a substantial risk in real-world settings. These poor success rates underscore a fundamental gap between current LLM capabilities and the reliable, stateful reasoning required for genuine autonomy.

7 Limitations

Our work has several limitations. First, we examine only a single domain. While the 8-puzzle is informative for tracking a model’s progress and may be representative of more challenging sequential planning tasks, it remains a single task, which limits our ability to draw broad conclusions about model performance in other settings. Second, we evaluate the models on only 50 puzzles, which is a relatively small dataset. Finally, although we generated puzzles at random to reduce the risk of training-data contamination, we cannot entirely rule it out.

References

- [1] OpenAI. GPT-5 system card. <https://openai.com/index/gpt-5-system-card/>, 2025. Web article; accessed 2025-11-10.
- [2] Gemini Team. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv*, 2025. URL <https://arxiv.org/abs/2507.06261>. arXiv:2507.06261.
- [3] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv*, 2020. URL <https://arxiv.org/abs/2009.03300>. arXiv:2009.03300.
- [4] Mislav Balunović, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. Matharena: Evaluating LLMs on uncontaminated math competitions. *arXiv*, 2025. URL <https://arxiv.org/abs/2505.23281>. arXiv:2505.23281.
- [5] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv*, 2023. URL <https://arxiv.org/abs/2310.06770>. arXiv:2310.06770.
- [6] Karthik Valmeekam, Kaya Stechly, Atharva Gundawar, and Subbarao Kambhampati. A systematic evaluation of the planning and scheduling abilities of the reasoning model o1. *Transactions on Machine Learning Research*, 2025. URL <https://openreview.net/forum?id=FkKBxp0FhR>. Reviewed on OpenReview.
- [7] François Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. ARC prize 2024: Technical report. *arXiv*, 2024. URL <https://arxiv.org/abs/2412.04604>. arXiv:2412.04604.
- [8] Marianna Nezhurina, Lucia Cipolina-Kun, Mehdi Cherti, and Jenia Jitsev. Alice in wonderland: Simple tasks showing complete reasoning breakdown in state-of-the-art large language models. *arXiv*, 2024. URL <https://arxiv.org/abs/2406.02061>. arXiv:2406.02061.
- [9] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating LLMs as agents. *arXiv*, 2023. doi: 10.48550/arXiv.2308.03688. URL <https://arxiv.org/abs/2308.03688>. arXiv:2308.03688.
- [10] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.
- [11] Zihan Yu, Liang He, Zhen Wu, Xinyu Dai, and Jiajun Chen. Towards better chain-of-thought prompting strategies: A survey. *arXiv*, 2023. URL <https://arxiv.org/abs/2310.04959>. arXiv:2310.04959.

- [12] Bilgehan Sel, Ahmad Al-Tawaha, Vanshaj Khattar, Ruoxi Jia, and Ming Jin. Algorithm of thoughts: Enhancing exploration of ideas in large language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, 2024.
- [13] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [14] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, 2023.
- [15] Boshi Wang, Sewon Min, Xiang Deng, Jiaming Shen, You Wu, Luke Zettlemoyer, and Huan Sun. Towards understanding chain-of-thought prompting: An empirical study of what matters. *arXiv*, 2022. URL <https://arxiv.org/abs/2212.10001>. arXiv:2212.10001.
- [16] Zhenni Bi, Kai Han, Chuanjian Liu, Yehui Tang, and Yunhe Wang. Forest-of-thought: Scaling test-time compute for enhancing LLM reasoning. *arXiv*, 2024. URL <https://arxiv.org/abs/2412.09078>. arXiv:2412.09078.
- [17] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Program-aided language models. In *Advances in Neural Information Processing Systems*, 2023.
- [18] Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*, 2023.
- [19] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv*, 2023. URL <https://arxiv.org/abs/2302.04761>. arXiv:2302.04761.
- [20] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv*, 2022. URL <https://arxiv.org/abs/2210.03629>. arXiv:2210.03629.
- [21] Mathematical Association of America. MAA invitational competitions. <https://maa.org/maa-invitational-competitions/>, 2025. Webpage; accessed 2025-08-26.
- [22] Long Phan, Alice Gatti, Ziwen Han, Nathaniel Li, Dan Hendrycks, Alexandr Wang, et al. Humanity’s last exam. *arXiv*, 2025. URL <https://arxiv.org/abs/2501.14249>. arXiv:2501.14249.
- [23] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhu Chen. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. *arXiv*, 2024. URL <https://arxiv.org/abs/2406.01574>. arXiv:2406.01574.
- [24] Mehran Kazemi, Bahare Fatemi, Hritik Bansal, John Palowitch, Chrysovalantis Anastasiou, Sanket Vaibhav Mehta, Lalit K. Jain, Virginia Aglietti, Disha Jindal, Peter Chen, Nishanth Dikkala, Gladys Tyen, Xin Liu, Uri Shalit, Silvia Chiappa, Kate Olszewska, Yi Tay, Vinh Q. Tran, Quoc V. Le, and Orhan Firat. BIG-Bench extra hard. *arXiv*, 2025. URL <https://arxiv.org/abs/2502.19187>. arXiv:2502.19187.
- [25] Cheng Xu, Shuhao Guan, Derek Greene, and M-Tahar Kechadi. Benchmark data contamination of large language models: A survey. *arXiv*, 2024. URL <https://arxiv.org/abs/2406.04244>. arXiv:2406.04244.
- [26] Shanchao Liang, Spandan Garg, and Roshanak Zilouchian Moghaddam. The SWE-Bench illusion: When state-of-the-art LLMs remember instead of reason. *arXiv*, 2025. URL <https://arxiv.org/abs/2506.12286>. arXiv:2506.12286.

- [27] Karthik Valmeekam, Sarath Sreedharan, Matthew Marquez, Alberto Olmo, and Subbarao Kambhampati. On the planning abilities of large language models (a critical investigation with a proposed benchmark). *arXiv*, 2023. URL <https://arxiv.org/abs/2302.06706>. arXiv:2302.06706.
- [28] Parshin Shojaei, Iman Mirzadeh, Keivan Alizadeh, Maxwell Horton, Samy Bengio, and Mehrdad Farajtabar. The illusion of thinking: Understanding the strengths and limitations of reasoning models via the lens of problem complexity. *arXiv*, 2025. URL <https://arxiv.org/abs/2506.06941>. arXiv:2506.06941.
- [29] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020.
- [30] Alexander Reinefeld. Complete solution of the eight-puzzle and the benefit of node ordering in IDA*. In *Proceedings of the 13th International Joint Conference on Artificial Intelligence (IJCAI-93)*, pages 248–253, Chambéry, France, 1993. Morgan Kaufmann. URL <https://www.ijcai.org/Proceedings/93-1/Papers/035.pdf>.
- [31] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. *arXiv*, 2022. URL <https://arxiv.org/abs/2205.11916>. arXiv:2205.11916.
- [32] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. SELF-REFINE: Iterative refinement with self-feedback. *arXiv*, 2023. URL <https://arxiv.org/abs/2303.17651>. arXiv:2303.17651.
- [33] OpenAI. Introducing GPT-5. <https://openai.com/index/introducing-gpt-5/>, 2025. Product page; accessed 2025-10-18.